

Memoria



Figura 1: Sintetizador

◆ Proyecto Sintetizador ◆ Sistemas Multimedia

Índice

♦ Introducción al Proyecto	3
♦ Requisitos Funcionales	4
→ Requisitos Mínimos	4
→ Requisitos Añadidos	4
♦ Metodología	6
♦ Diseño de la Interfaz	7
♦ Implementación	11
→ Plugin Processor	11
→ Plugin Editor	12
→ Synth Voice	13
♦ Instalación	14

◆ Introducción al Proyecto

El proyecto escogido para desarrollar las prácticas de la asignatura de Sistemas Multimedia ha sido la realización de un instrumento virtual, un sintetizador. Primeramente, se ha de definir lo que es un sintetizador y cuáles son sus características para comprender cuál ha sido el objetivo principal de este trabajo.

Según la definición de la RAE, un **sintetizador** es “*un instrumento musical electrónico capaz de producir sonidos de cualquier frecuencia e intensidad y combinarlos con armónicos, proporcionando así sonidos de cualquier instrumento conocido, o efectos sonoros que no corresponden a ningún instrumento convencional*”. De acuerdo a esta definición, un sintetizador es un instrumento musical digital capaz de emular las ondas de instrumentos acústicos, o generar las suyas propias gracias a la síntesis y computación puesto que es capaz de transformar e interpretar una onda definida matemáticamente. Aunque un sintetizador no es únicamente un intérprete de ondas, está compuesto principalmente por un Oscilador y un Filtro. Los osciladores son los encargados de producir la síntesis de las ondas, y los filtros dejan pasar ciertas frecuencias de dichas ondas.

Para el desarrollo de la práctica busqué herramientas y guías de cómo crear un instrumento virtual. En el mundo del desarrollo de instrumentos virtuales, plug-ins, y DAWs (Digital Audio Workstations), era bastante común el uso de C++, y encontré JUCE.

JUCE es un framework enfocado al desarrollo de estas aplicaciones de audio, es de código abierto, multiplataforma y escrito en C++. Con la licencia estándar y gratuita te permite empezar a desarrollar cualquier variante de aplicación de audio. Tan sólo se requiere licencia para comercializar los productos. Dispone de mucha documentación acerca de sus módulos y clases, e incluso tutoriales para familiarizarte con el framework.

◆ Requisitos Funcionales

Consideré que los requisitos para esta aplicación se podrían dividir en dos categorías desde lo más básico a lo más prescindible.

→ Requisitos Mínimos

- Oscilador
- Entrada/Salida MIDI
- Regulador de Volumen
- Formato VST3 compatible con cualquier DAW

Estos requisitos son esenciales para que se considere el proyecto un sintetizador simple y funcional. Un oscilador encargado de la síntesis y producción de la onda. Capaz de que se pueda controlar con entrada y salida MIDI, esencial para cualquier instrumento virtual actual. Un regulador de volumen, que controle la ganancia de la onda y que el usuario pueda determinar el volumen que quiera. Y por último, como elemento esencial, que se pueda trasladar el proyecto como un instrumento virtual de verdad a formato VST3, que es el estándar actual en la industria de plug-ins. De esta forma se podría usar esta aplicación en diferentes DAWs y ámbitos de la producción musical.

→ Requisitos Añadidos

- Muestra Visual de la Onda
- Regulador de Paneo
- Filtros (Paso Bajo, Paso Alto, Banda)
- Frecuencia Modulada
- ADSR (Attack, Decay, Sustain, Release)
- ADSR modulado
- Piano Visual
- Reverb
- Delay
- Ecualizador o Compresor
- Varios Osciladores

Por otra parte, están los requisitos añadidos, que se consideran un plus o que complementan al sintetizador principal. El primero, la muestra visual de la onda, me pareció una forma interesante de enseñar al usuario lo que está ocurriendo a nivel de sonido de una forma más directa e intuitiva. Luego está el regulador de paneo, que podría considerarse como elemento esencial, pero la decisión de que fuese un añadido vino porque la gran mayoría de DAWs ya lo tienen incorporado y podrían sustituirlo. A

continuación, están los filtros, la frecuencia modulada y los ADSRs, que modifican la onda y afectan a sus frecuencias directamente. Son una forma de añadir personalización y exploración de sonidos al usuario. Al igual que la onda visual, está el requisito de un piano que mostrase las teclas o notas que se están tocando al momento, con el fin de informar o retroalimentar al usuario de lo que está pasando. Finalmente, se encuentran los requisitos más prescindibles puesto que se podrían conseguir de otras formas externas al sintetizador en sí, como son la *Reverb*, *Delay*, *Ecualizador*, *Compresor* o la suma de varios osciladores.

Finalmente, como requisitos no funcionales, pero sí que podrían tratarse de esenciales en cuanto al desarrollo de este proyecto, se trata de conseguir una buena usabilidad para el usuario. Esto se debería de conseguir teniendo en cuenta factores como la efectividad, la eficiencia y la satisfacción al usar la aplicación. Aunque este trabajo está enfocado hacia usuarios con algo de experiencia y que pueden entender más o menos los conceptos con los que se va a trabajar, los elementos que se presentan deben de ser fáciles de manejar y de aprender, e intuitivos como pilar base del proyecto. Otro factor importante para la usabilidad, es la apariencia, el diseño de la interfaz, del cual se hablará más adelante. Se debe presentar un diseño simple, que no sea abrumador y dificulte la tarea al usuario, que el texto y la tipografía tenga el tamaño adecuado para que se pueda leer bien, y por último, que la interacción con los elementos sea lo más fluida posible, dando varias posibilidades al usuario de manejarlos y controlarlos, como puede ser arrastrar los controles o con la rueda del ratón.

♦ Metodología

Para el desarrollo del proyecto he seguido la metodología ágil SCRUM aunque algo modificada debido a que no hay un equipo para el desarrollo y las reuniones o sprints se han traducido en presentar el progreso del trabajo en las horas de prácticas semanales.

Primeramente, se realizó un diseño de la interfaz de usuario que presentaría la aplicación y los correspondientes requisitos funcionales para un sintetizador. Una vez recabada esta información, empezó la fase de desarrollo y aprendizaje del framework.

Las primeras semanas de desarrollo se enfocaron a conseguir los requisitos básicos o mínimos previamente explicados. Mediante prueba, errores, refactorización de código, lectura básica de la documentación y tutoriales iniciales, se obtuvo una versión inicial del sintetizador, en la que se podía reproducir una onda sinusoidal pulsando cualquier tecla del teclado del ordenador traduciéndose a una nota musical aplicando la frecuencia correspondiente a la onda. Con esa versión funcional, se pudo progresar a las siguientes etapas añadiendo funcionalidades.

A cada tarea se debía comprobar que lo último añadido no dañara o rompiera lo conseguido hasta el momento. En varias ocasiones, al implementar un nuevo efecto o filtro, daba errores de compilación en las librerías del framework, errores de punteros y fallos de acceso a memoria. La mayoría de las veces se solucionaban fácilmente, pero otras, se tuvo que recurrir a crear un proyecto nuevo y copiar archivos, y en un caso más complicado, restablecer las librerías y módulos del framework a su versión original porque en un proceso de compilación se habían corrompido.

Una vez alcanzados los requisitos mínimos del proyecto, dio comienzo la fase de implementar los requisitos añadidos que podían hacer más interesante el desarrollo del trabajo. En esta fase se dedicó el tiempo a buscar ejemplos, referencias o directrices para desarrollar la muestra visual de la onda, el efecto de *reverb*, o incluso añadir otro oscilador para sumar ambas ondas y obtener un sonido más personalizado. Este último reto no pudo llevarse a cabo dado que los errores que aparecían no pudieron solucionarse en el tiempo estimado para su desarrollo y se tuvo que optar por desechar esa idea y continuar con otros objetivos planteados.

Finalmente, en la última etapa, con el producto final totalmente funcional, se procedió al diseño de la interfaz gráfica y de usuario. Se obtuvo un resultado satisfactorio y decente en comparación con otros instrumentos virtuales o plug-ins ya existentes en el mercado. Por tanto, se dio por finalizado el proyecto.

◆ Diseño de la Interfaz

El desarrollo de la interfaz se mantuvo en paralelo y en las primeras fases del trabajo algo relegada o dependiente de las funcionalidades. Un primer boceto de lo que podría ser el proyecto en su versión final era este:

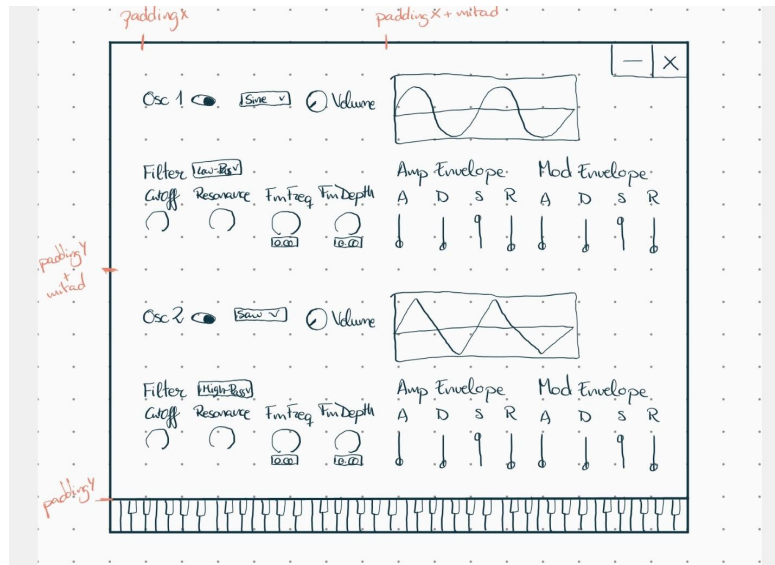


Figura 2: Boceto inicial del proyecto

En este primer diseño, aparecían los dos osciladores con sus ondas visuales y diferentes parámetros y efectos, y en la parte inferior un piano que mostraría las teclas que se están pulsando o tocando. Pero poco se parecerían las primeras versiones del proyecto con ese diseño, como se puede ver en la siguiente imagen.



Figura 3: Imagen de las primeras versiones del proyecto

Una vez todos los elementos que conforman el sintetizador funcionaban perfectamente, se procedió a realizar el diseño y recolocar dichos elementos. Primeramente, aparecería el nombre del proyecto y justo debajo la muestra visual de la onda. Se quiso destacar este elemento visual, puesto que sería lo más atractivo para el usuario final.

Después, viene la colocación del resto de elementos, desde los más fundamentales hasta los efectos complementarios. Por un lado está el oscilador principal, con la selección en formato desplegable del tipo de onda, Sinusoidal, de Sierra, o Cuadrada. Y unos *sliders* o controles deslizantes para los parámetros de la frecuencia modal. A su derecha, y en un menor tamaño, los controles de volumen y panning. Con panning se refiere a la cantidad de sonido que recibe un monitor derecho o izquierdo en un formato estéreo. A su lado se encuentra el filtro principal, con un desplegable al igual que el oscilador para elegir entre filtro de Paso-Bajo, Paso-Alto, o de Banda, y además unos controles para elegir a qué frecuencias se quiere afectar y la resonancia a aplicar.

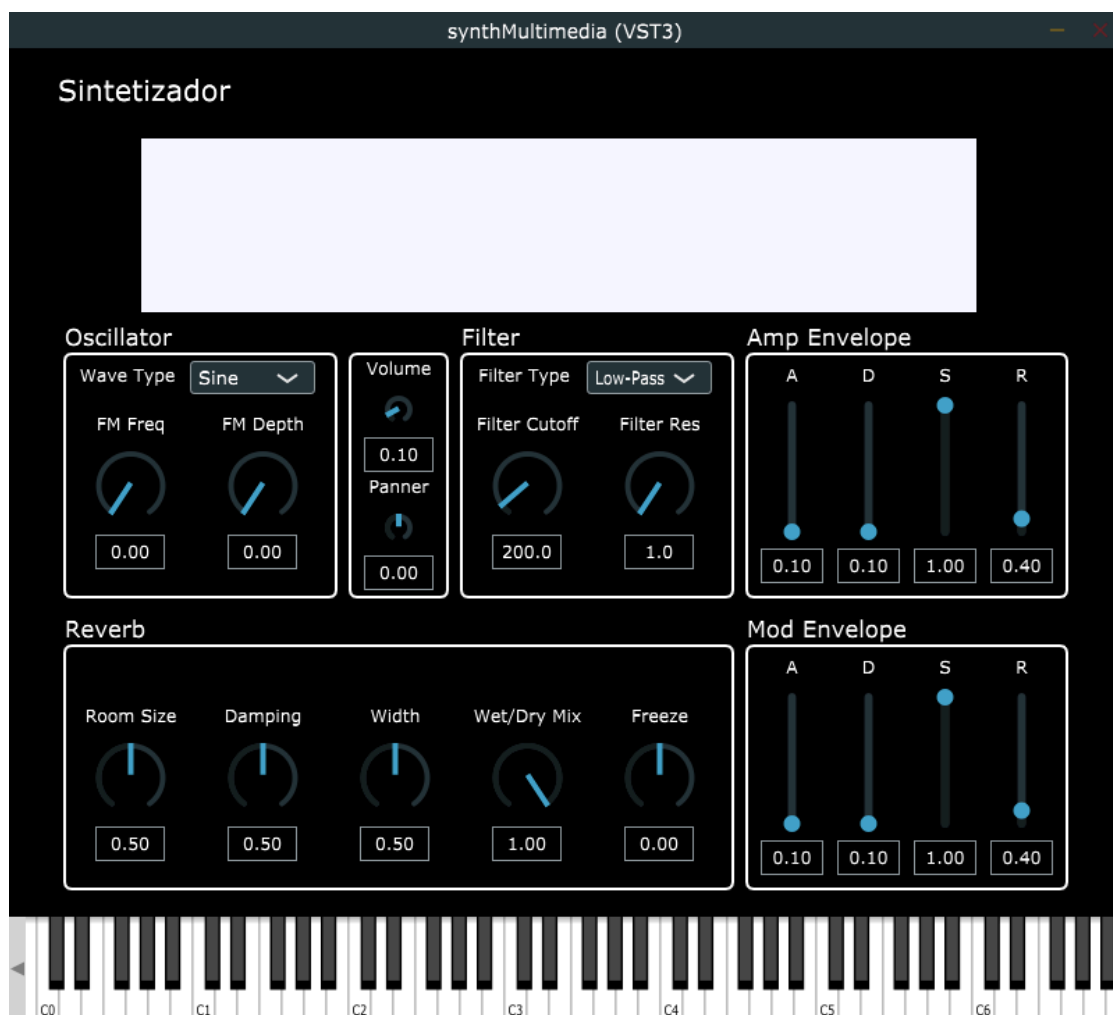


Figura 4: Versión funcional final del proyecto

Luego están los controles de ADSR (Attack, Decay, Sustain, Release) para las ondas del oscilador y la frecuencia modulada. Estos controles se ha decidido que fueran deslizantes verticalmente, puesto que se consideró que eran más intuitivos para el usuario. Y por último, el efecto de *reverb*, en la que se encuentran varios controles dedicados a el tamaño de la habitación en la que rebotaría el sonido, la humedad en dicha habitación, la anchura, el Wet/Dry se refiere a la cantidad de reverberación que se quiere aplicar a la onda, y freeze mode en la que se puede congelar o mantener en loop la reverberación.

Con este formato en cajas separadoras, el siguiente paso fue la elección de colores para la interfaz. Atendiendo a las pautas y directrices del uso efectivo del color, se optó por una paleta de unos 4 colores principales.

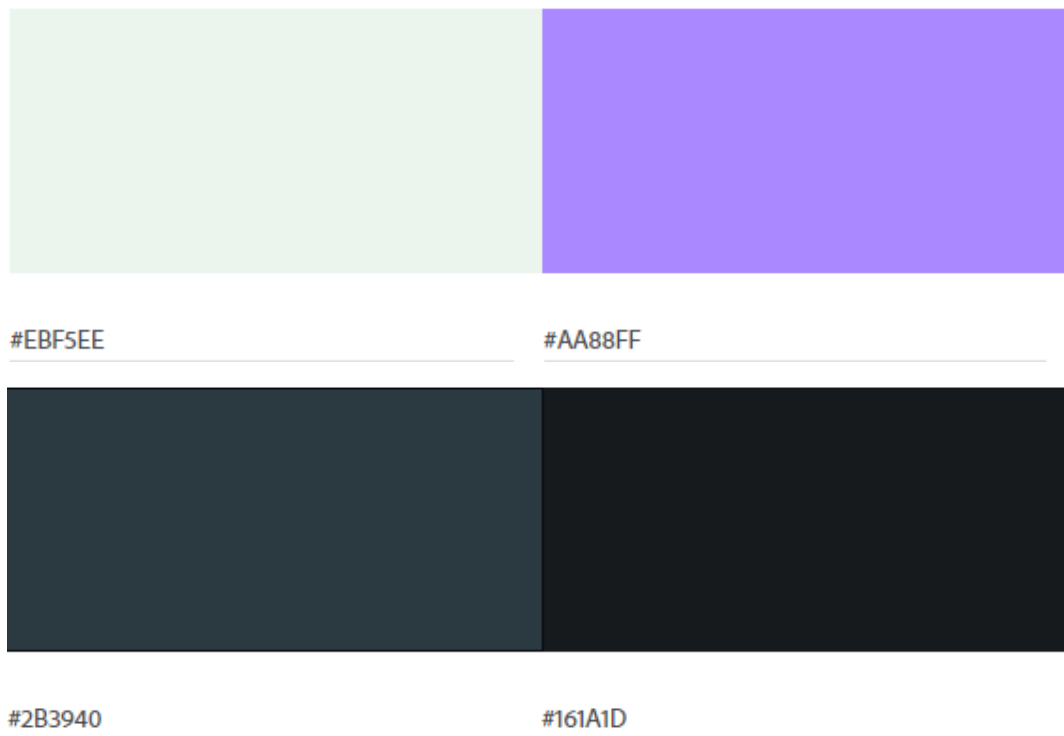


Figura 5: Paleta de colores y sus respectivos códigos

Para el fondo de la aplicación se usó el color más oscuro, sin llegar a ser un color negro absoluto como #FFFFFF, que se había aplicado en versiones anteriores del diseño. Mientras que para las letras, el texto, los botones, y los recuadros de cada funcionalidad, se eligió un color blanco crema, con la misma filosofía de no escoger un blanco puro, y obtener una apariencia más suave. Y por último, para los controles de los parámetros, se determinó el color principal violeta o lavanda, como se puede ver en la muestra visual de la onda o en los controles según la cantidad aplicada, en contraposición al gris, recreando así un efecto de relleno.



Figura 6: Diseño final del sintetizador

Así quedaría el proyecto en su versión final del diseño, aplicando los colores, tamaños y posiciones.

◆ Implementación

En esta sección, se explicará cómo se ha llevado a cabo el proyecto en términos de programación e implementación. Lo primero de todo a destacar, es que este proyecto ha sido desarrollado en el lenguaje C++, y en mi caso particular, utilizando Visual Studio. Se pueden utilizar otras aplicaciones como Xcode para el caso de MAC o iOS, Linux Makefile, Android, o incluso CodeBlocks. Según la aplicación escogida, se deben de seguir unos pasos para poder probar y debuggear la aplicación a lo largo de su desarrollo.

El código de la aplicación se compone principalmente de:

- **Plugin Processor:** se encarga de montar la aplicación y controlar los parámetros que recibe de entrada y gestionarlos a su salida.
- **Plugin Editor:** su propósito principal es de manejar todos los elementos de la aplicación y distribuirlos en la interfaz.
- **Synth Sound:** es el sonido principal que se genera al construir el sintetizador, puesto que este está compuesto por varias voces que pueden sonar a la vez.
- **Synth Voice:** es el elemento principal de este trabajo, es el sintetizador. Gestiona los datos de entrada que recibe por parte del procesador y hace las modificaciones necesarias al sintetizador

Luego, se ha optado por dividir en carpetas el código según su propósito. Por una parte, los ficheros encargados del procesamiento de los datos entre entrada y salida que se encuentran en la carpeta de `/Data`. Y por otra parte, los archivos que contienen los componentes gráficos que gestionan cada uno de los elementos de las funcionalidades en la representación gráfica de la aplicación, dentro de la carpeta `/UI`.

En cada archivo se definen una serie de métodos necesarios para el completo funcionamiento de la aplicación, pero será mejor destacar los más importantes y fundamentales.

→ Plugin Processor

En esta clase cabe destacar cuatro métodos esenciales para cualquier aplicación o plug-in. Primeramente, el constructor, donde se debe crear la instancia del APVTS, el [Audio Processor Value Tree State](#). Es el encargado de gestionar todos los parámetros de entrada y salida de la interacción entre componentes y el usuario. Luego se configura el visualizador de onda, determinando las muestras por bloque y el tamaño del buffer, recomendable que sean potencias de 2. También se restablece el estado del piano para que no haya ningún error. Y finalmente, se inicializan las voces del sintetizador.

El siguiente método, *prepareToPlay*, se establece la ratio de muestras, y las muestras por bloque que será capaz el sintetizador de procesar. Se hace este proceso por cada voz existente que tenga el sintetizador en ese momento. Y lo último en este método, se limpia el visualizador de ondas.

El tercer método, *processBlock*, se limpia el buffer de audio, y para cada voz del sintetizador se les pasan los parámetros definidos en el APVTS, para que la clase *SynthVoice* se encargue de procesarlos debidamente. Después, se crea la muestra visual tanto del piano como de la onda, y se llama al método del sintetizador para renderizar el siguiente bloque.

Y por último, el cuarto método, *createParams*, está dedicado a definir y crear los parámetros del APVTS, introduciéndolos en un vector. Para cada parámetro, se le dan unos IDs únicos, una descripción, unos valores predeterminados como es el caso de la elección de ondas o de filtros, o un rango de valores con un incremento también previamente definido. En ciertos parámetros se define un *Skew Factor*, el cual ofrece mayor control sobre los valores más pequeños en comparación a los valores más grandes, puesto que el incremento es logarítmico.

→ Plugin Editor

En el editor se necesitan los componentes gráficos de cada elemento del sintetizador para mostrarlos por pantalla, el procesador de audio, y por último una clase personalizada para dar el estilo a los controles deslizantes rotatorios en vez de renderizar los que vienen por defecto en JUCE.

Al igual que en el Plugin Processor, el Plugin Editor dispone de otros tres métodos fundamentales. En el constructor, se deben inicializar los atributos de los componentes, pasándoles los identificadores únicos definidos en el APVTS y el procesador de audio. Se debe comprobar que son los mismos identificadores, escritos exactamente igual, puesto que puede ocasionar problemas. A continuación, se define el *Look and Feel* personalizado, que luego debe de destruirse en el destructor, se define el tamaño de la pantalla de la aplicación, los colores de las teclas del piano o de las ondas, y por último hacer visibles los componentes.

En el método *paint* los colores generales del proyecto, y en este caso también títulos, tamaño de letra y color de fuente. Mientras que en el último método, *resized*, se determinan las posiciones exactas de cada uno de los componentes.

→ Synth Voice

Esta clase deriva de la definida por JUCE, *SynthesiserVoice*, pero se redefinen los métodos principales para agregarles el resto de componentes. Recibe como atributos todas las clases definidas en la carpeta /Data y las actualiza cuando el Plugin Processor le pasa toda la información de los parámetros.

Los métodos *startNote* y *stopNote*, permiten o no que se transfieran los datos al oscilador para que comience a sonar según la nota MIDI, o que se activen o desactiven los efectos producidos por los ADSRs.

En el método *prepareToPlay*, se inicializan todos los componentes pasándoles la ratio de muestras, las muestras por bloque, y los canales de salida. Una vez todo ha ido correctamente, se puede decir que el sintetizador está preparado.

El siguiente método, *renderNextBlock*, se comprueba que el sintetizador está listo. Si es así, se procede a inicializar el buffer y mandándoles a los componentes el buffer para que procesen su información.

Los métodos a continuación, son los que se encargan de actualizar las variables y parámetros de cada componente. Pero al ser tan triviales, no necesitan de una explicación detallada.

◆ Instalación

Para la instalación de este proyecto se debe atender a dos objetivos. Si se quiere tan solo como producto final y probar en un DAW a elección. O por el contrario, si se quiere ver y probar con el código proporcionado.

En cuanto a la primera opción, el producto final del proyecto es el archivo `synthMultimedia.vst3`. Dando por hecho que se dispone de un DAW, este archivo debe ser colocado en la carpeta principal donde se encuentran el resto de VST3 del dispositivo. En Windows suelen estar en `C:\Program Files\Common Files\VST3`. Si el dispositivo es MAC, la ruta sería la siguiente `MacintoshHD/Library/Audio/Plug-Ins/VST3`. En el caso de Linux, puede que la carpeta se encuentre en `/home/user/vst`. Una vez ahí, el DAW debería de ser capaz de reconocerlo entre sus instrumentos virtuales disponibles. Si no fuese así, se debería de probar a actualizar o refrescar los plug-ins detectados por los DAW.

El archivo `.vst3` del proyecto se ha probado en tres diferentes DAWs: Ableton, Reaper y LMMS. Siendo este último de código abierto y disponible para cualquier plataforma. Sin embargo, es el que más problemas ha dado para reconocer archivos VST3, puesto que solo reconoce librerías `.dll`. Así que ha hecho falta otro software de código abierto para que haga de intermediario y se pueda abrir el instrumento virtual en LMMS. Ese software ha sido [Element](#), de Kush View. Para los otros dos DAWs, tan solo ha hecho falta abrir el buscador de plug-ins e instrumentos virtuales, y en el apartado de *youcompany*, que así denomina JUCE a los productos sin licencia por defecto, se encontraría el sintetizador `synthMultimedia.vst3`.

Para la segunda opción de instalación y prueba, se debe descargar el framework de [JUCE](#) para la plataforma deseada. Una vez descomprimida la carpeta de instalación, dentro se encontrará la aplicación Projucer, y desde ahí podrá abrir el código del proyecto siguiendo el mismo procedimiento que en cualquier otra aplicación. Por defecto, se abrirá en Visual Studio 2019, pero si quiere cambiar de aplicación, en la parte izquierda de la pantalla, en Exporters, encontrará un símbolo de + para añadir la aplicación que desee para abrir el proyecto. Una vez dentro del proyecto, se puede navegar por los distintos archivos de código y ver cómo está estructurado. En el caso que se quiera modificar código y posteriormente probarlo, hay que seguir una serie de pasos según la aplicación de edición de código que se use.

Si es el caso de Visual Studio, hay que ir al apartado de Depurar y entrar en “Propiedades de la Depuración de `synthMultimedia`”, y dentro de la ventana, ir al apartado de Depuración. Allí habrá que rellenar el apartado de Comando con la ruta de

AudioPluginHost, incluido en la aplicación de JUCE, de tal forma que la ruta sería la siguiente:

JUCE\extras\AudioPluginHost\Builds\VisualStudio2019\x64\Debug\App\AudioPluginHost.exe

Para el caso de utilizar Xcode, en este [tutorial](#) de JUCE se explica cómo configurar la opción de debuggear el proyecto al igual que con Visual Studio.

Esas serían las opciones de configuración e instalación para abrir y ejecutar el proyecto del sintetizador.